Lecture 04: Control Flow, Numbers, Math and Strings

6. Bitwise Operators

They perform operations directly on the binary representation (bits) of numbers.

Operator	Name	Description
&	Bitwise AND	1 if both bits are 1
|	Bitwise OR	1 if at least one of the bits is 1
^	Bitwise XOR	1 if both bits are different
~	Bitwise NOT	Invert all bits
<<	Left Shift	Shift bits to the left
>>	Right Shift	Shift bits to the right, preserves sign bit
>>>	Unsigned Right Shift	Shift bits right and fill left with 0

Bitwise AND

```
  00001000
& 00000101
----------
  00000000
```

↓

$(0)_{10}$

Bitwise OR

```
  00001000
| 00000101
----------
  00001101
```

↓

$(13)_{10}$

Bitwise XOR

```
  00001000
^ 00000101
----------
  00001101
```

↓

$(13)_{10}$

Bitwise NOT

8 → 00001000 —Bitwise NOT→ 11110111 —2's Complement→ 00001001

↓

$(-9)_{10}$

Because it was a negative number with a magnitude of 9, the final decimal value is: -9

Left Shift

8 → 00001000

↓

00010000

↓

$(16)_{10}$

Right Shift

8 → 00001000

↓

00000100

↓

$(4)_{10}$

The Classical Floating-Point Problem:

JavaScript uses the IEEE 754 double-precision standard. Certain decimal fractions cannot be represented exactly in binary, leading to precision loss.

```
console.log(0.1 + 0.2);
Output: 0.30000000000000004
```

Conversion of Decimal Fractions to Binary

0.1 × 2 = 0.2 → 0
0.2 × 2 = 0.4 → 0
0.4 × 2 = 0.8 → 0

0.8 × 2 = 1.6 → 1
0.6 × 2 = 1.2 → 1
0.2 × 2 = 0.4 → 0 (The pattern starts repeating)

Best Practice for Financial Data: Financial data should never be stored as decimals. Instead, values must be stored in their smallest integer unit (e.g., storing paise instead of rupees, or Satoshis instead of Bitcoin) to entirely avoid precision errors during calculations.

Conditional Statements:

Conditional statements execute different blocks of code based on specific conditions.

1. if Statement

Executes a block of code only if the condition evaluates to true.

Syntax:

```
if (condition) {
  // This code runs only if the condition is true.
}
```

Example:

```
if(age >= 18) {
   console.log("Eligible to vote.");
}
```

2. if-else Statement

Provides a fallback block of code if the initial condition is false.

Syntax:

```
if (condition) {
  // This code runs if the condition is true.
} else {
  // This code runs if the condition is false.
}
```

Example:

```
if(num >= 0) {
   console.log("Number is positive.");
} else {
   console.log("Number is negative.");
}
```

3. if-else-if Ladder

Evaluates multiple conditions in sequence. The chain stops as soon as the first true condition is met.

Syntax:

```
if (condition1) {
  // Runs if condition1 is true.
} else if (condition2) {
  // Runs if condition1 is false and condition2 is true.
} else if (condition3) {
  // Runs if 1 and 2 are false and condition3 is true.
} else {
  // Runs if all previous conditions were false.
}
```

Example:

```
if(marks >= 90) {
   console.log("Grade A");
} else if(marks >= 90) {
   console.log("Grade B");
} else if(marks >= 50) {
   console.log("Grade C");
} else {
   console.log("Fail");
}
```

Loops:

A loop executes a block of code repeatedly for a predefined number of times or until a condition becomes false.

1. for loop

A for loop is used to execute a block of code repeatedly when the number of iterations is known.

Syntax:

```
for (initialization; condition; update) {
   // statements
}
```

Example:

```
for(let i = 1; i <= 10; i++) {
   console.log(i);
}
```

2. while loop

The while loop checks the condition first and then executes the code. It is an "entry-controlled" loop.

Syntax:

```
while (condition) {
   // code
}
```

Example:

```
let i = 1;
while(i <= 10) {
   console.log(i);
   i++;
}
```

3. do-while loop

The do-while loop executes the code at least once, then checks the condition. It is an "exit-controlled" loop.

Syntax:

```
do {
   // code
} while (condition);
```

Example:

```
let i = 1;
do{
   console.log(i);
   i++;
}while(i <= 10);
```

Numbers in JavaScript:

- Numbers are a primitive data type used to represent both integers and floating-point values.
- JavaScript doesn't have separate types for int, float, double.

Creating Numbers:

1. Standard Literals

```
let a = 10;
let b = 3.14;
```

2. Object Constructor

```
let num2 = new Number (10);
```

Creates an object, not a primitive. Highly discouraged as it impacts performance and comparison operations.

Special Numeric Values:

Infinity: Represents mathematical infinity.
-Infinity: Represents negative mathematical infinity.
NaN (Not a Number): A computational error indicating a failed mathematical operation.

Key Methods & Properties:

Method	Description
isNaN(x)	Returns true if the value is not a valid number.
.toFixed(x)	Formats a number to a fixed number of decimal places and returns a string.
.toPrecision(x)	Formats a number to a specified total length and returns a string.
.toString(x)	Converts a number into its string representation.

The Math Object:

The Math object provides built-in mathematical constants and functions to perform mathematical operations.

1. Constants: Math.PI, Math.E

2. Rounding:

- Math.round(x): Standard rounding to the nearest integer.
- Math.floor(x): Rounds down to the nearest integer.
- Math.ceil(x): Rounds up to the nearest integer.
- Math.trunc(x): Removes the decimal part, leaving only the integer (truncates toward zero).

3. Other Common Functions:

- Math.abs(x): Absolute value.
- Math.pow(x, y): x to the power of y.
- Math.sqrt(x): Square root.
- Math.cbrt(x): Cube root.
- Math.max(x, y, z...): Returns the largest of the given numbers.
- Math.min(x, y, z...): Returns the smallest of the given numbers.
- Math.random(): Returns a pseudo-random number between 0 (inclusive) and 1 (exclusive).

Warning: Math.random() is not cryptographically secure and should not be used for high-security applications like OTP generation, as its output is deterministic and predictable.

Note: OTP services often use algorithms that may link OTP generation to phone numbers or session IDs to ensure uniqueness or verification, rather than relying on standard pseudo-random number generators.

Strings:

A string is a primitive data type in JavaScript used to represent a sequence of characters.

Creating Strings:

Standard Strings ("" or ''): Do not support multi-line text directly.

```
let str1 = "Rohit Negi";
let str2 = 'Hello Coder Army';
```

Template Literals (`): Support multi-line strings without needing special escape characters.

```
let str3 = `This is a template literal.`;
```

It allows embedding variables and expressions directly into the string using ${variableName} syntax.

Key Methods & Properties:

Method	Description
.length	Returns the total count of characters.
.toLowerCase()	Returns a new string converted to lowercase.
.toUpperCase()	Returns a new string converted to uppercase.
.indexOf(sub)	Returns the index of the first occurrence of a substring, returns -1 if not found.
.lastIndexOf(sub)	Returns the index of the last occurrence of a substring. Returns -1 if not found.
.includes(sub)	Returns a boolean. This is often more readable than indexOf.
.slice(start, end)	Extracts a section of a string and returns it as a new string.
.substring(start, end)	Similar to slice, but does not support negative indices.
.replace(searchValue, newValue)	Replaces the first occurrence, returns a new string with the value(s) replaced.
.replaceAll(searchValue, newValue)	Replaces all occurrences, returns a new string with all values replaced.